JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Class ByteArrayInputStream

java.lang.Object
    java.io.InputStream
        java.io.ByteArrayInputStream

**All Implemented Interfaces:**

Closeable, AutoCloseable

---

```
public class ByteArrayInputStream
extends InputStream
```

A `ByteArrayInputStream` contains an internal buffer that contains bytes that may be read from the stream. An internal counter keeps track of the next byte to be supplied by the `read` method.

Closing a `ByteArrayInputStream` has no effect. The methods in this class can be called after the stream has been closed without generating an `IOException`.

**Since:**

1.0

**See Also:**

StringBufferInputStream

## *Field Summary*

**Fields**

| Modifier and Type | Field | Description |
|---|---|---|
| `protected byte[]` | **buf** | An array of bytes that was provided by the creator of the stream. |
| `protected int` | **count** | The index one greater than the last valid byte in the input stream buffer. |
| `protected int` | **mark** | The currently marked position in the stream. |
| `protected int` | **pos** | The index of the next byte to read from the input stream buffer. |

## Constructor Summary

**Constructors**

| Constructor | Description |
|---|---|
| `ByteArrayInputStream(byte[] buf)` | Creates a `ByteArrayInputStream` so that it uses `buf` as its buffer array. |
| `ByteArrayInputStream(byte[] buf, int offset, int length)` | Creates `ByteArrayInputStream` that uses `buf` as its buffer array. |

## Method Summary

**All Methods** | **Instance Methods**

| Modifier and Type | Method | Description |
|---|---|---|
| `int` | `available()` | Returns the number of remaining bytes that can be read (or skipped over) from this input stream. |
| `void` | `close()` | Closing a `ByteArrayInputStream` has no effect. |
| `void` | `mark (int readAheadLimit)` | Set the current marked position in the stream. |
| `boolean` | `markSupported()` | Tests if this `InputStream` supports mark/reset. |
| `int` | `read()` | Reads the next byte of data from this input stream. |
| `int` | `read(byte[] b, int off, int len)` | Reads up to `len` bytes of data into an array of bytes from this input stream. |
| `byte[]` | `readAllBytes()` | Reads all remaining bytes from the input stream. |

| | | |
|---|---|---|
| int | **readNBytes**(byte[] b, int off, int len) | Reads the requested number of bytes from the input stream into the given byte array. |
| void | **reset**() | Resets the buffer to the marked position. |
| long | **skip**(long n) | Skips n bytes of input from this input stream. |

**Methods declared in class InputStream**

nullInputStream, read, readNBytes, skipNBytes, transferTo

**Methods declared in class Object**

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

## *Field Details*

### buf

```
protected byte[] buf
```

An array of bytes that was provided by the creator of the stream. Elements `buf[0]` through `buf[count-1]` are the only bytes that can ever be read from the stream; element `buf[pos]` is the next byte to be read.

### pos

```
protected int pos
```

The index of the next byte to read from the input stream buffer. This value should always be nonnegative and not larger than the value of `count`. The next byte to be read from the input stream buffer will be `buf[pos]`.

### mark

```
protected int mark
```

The currently marked position in the stream. ByteArrayInputStream objects are marked at position zero by default when constructed. They may be marked at another position within the buffer by the `mark()` method. The current buffer position is set to this point by the `reset()` method.

If no mark has been set, then the value of mark is the offset passed to the constructor (or 0 if the offset was not supplied).

**Since:**

1.1

### count

```
protected int count
```

The index one greater than the last valid byte in the input stream buffer. This value should always be nonnegative and not larger than the length of `buf`. It is one greater than the position of the last byte within `buf` that can ever be read from the input stream buffer.

## *Constructor Details*

### ByteArrayInputStream

```
public ByteArrayInputStream(byte[] buf)
```

Creates a `ByteArrayInputStream` so that it uses `buf` as its buffer array. The buffer array is not copied. The initial value of `pos` is `0` and the initial value of `count` is the length of `buf`.

**Parameters:**

`buf` - the input buffer.

### ByteArrayInputStream

```
public ByteArrayInputStream(byte[] buf,
                            int offset,
                            int length)
```

Creates `ByteArrayInputStream` that uses `buf` as its buffer array. The initial value of `pos` is `offset` and the initial value of `count` is the minimum of `offset+length` and `buf.length`. The buffer array is not copied. The buffer's mark is set to the specified offset.

**Parameters:**

`buf` - the input buffer.

`offset` - the offset in the buffer of the first byte to read.

`length` - the maximum number of bytes to read from the buffer.

## *Method Details*

### read

```
public int read()
```

Reads the next byte of data from this input stream. The value byte is returned as an `int` in the range `0` to `255`. If no byte is available because the end of the stream has been reached, the value `-1` is returned.

This `read` method cannot block.

**Specified by:**

`read` in class `InputStream`

**Returns:**

the next byte of data, or `-1` if the end of the stream is reached.

### read

```
public int read(byte[] b,
                int off,
                int len)
```

Reads up to `len` bytes of data into an array of bytes from this input stream. If `pos` equals `count`, then `-1` is returned to indicate end of file. Otherwise, the number `k` of bytes read is equal to the smaller of `len` and `count-pos`. If `k` is positive, then bytes `buf[pos]` through `buf[pos+k-1]` are copied into `b[off]` through `b[off+k-1]` in the manner performed by `System.arraycopy`. The value `k` is added into `pos` and `k` is returned.

Unlike the `overridden method` of `InputStream`, this method returns `-1` instead of zero if the end of the stream has been reached and `len == 0`.

This `read` method cannot block.

**Overrides:**

`read` in class `InputStream`

**Parameters:**

`b` - the buffer into which the data is read.

`off` - the start offset in array `b` at which the data is written.

`len` - the maximum number of bytes to read.

**Returns:**

the total number of bytes read into the buffer, or `-1` if there is no more data because the end of the stream has been reached.

**Throws:**

`NullPointerException` - If `b` is `null`.

`IndexOutOfBoundsException` - If `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

**See Also:**

`InputStream.read()`

## readAllBytes

```
public byte[] readAllBytes()
```

**Description copied from class: `InputStream`**

Reads all remaining bytes from the input stream. This method blocks until all remaining bytes have been read and end of stream is detected, or an exception is thrown. This method does not close the input stream.

When this stream reaches end of stream, further invocations of this method will return an empty byte array.

Note that this method is intended for simple cases where it is convenient to read all bytes into a byte array. It is not intended for reading input streams with large amounts of data.

The behavior for the case where the input stream is *asynchronously closed*, or the thread interrupted during the read, is highly input stream specific, and therefore not specified.

If an I/O error occurs reading from the input stream, then it may do so after some, but not all, bytes have been read. Consequently the input stream may not be at end of stream and may be in an inconsistent state. It is strongly recommended that the stream be promptly closed if an I/O error occurs.

**Overrides:**

`readAllBytes` in class `InputStream`

**Returns:**

a byte array containing the bytes read from this input stream

## readNBytes

```
public int readNBytes(byte[] b,
                      int off,
                      int len)
```

**Description copied from class: `InputStream`**

Reads the requested number of bytes from the input stream into the given byte array. This method blocks until `len` bytes of input data have been read, end of stream is detected, or an exception is thrown. The number of bytes

actually read, possibly zero, is returned. This method does not close the input stream.

In the case where end of stream is reached before `len` bytes have been read, then the actual number of bytes read will be returned. When this stream reaches end of stream, further invocations of this method will return zero.

If `len` is zero, then no bytes are read and `0` is returned; otherwise, there is an attempt to read up to `len` bytes.

The first byte read is stored into element `b[off]`, the next one in to `b[off+1]`, and so on. The number of bytes read is, at most, equal to `len`. Let *k* be the number of bytes actually read; these bytes will be stored in elements `b[off]` through `b[off+`*k*`-1]`, leaving elements `b[off+`*k*` ]` through `b[off+len-1]` unaffected.

The behavior for the case where the input stream is *asynchronously closed*, or the thread interrupted during the read, is highly input stream specific, and therefore not specified.

If an I/O error occurs reading from the input stream, then it may do so after some, but not all, bytes of `b` have been updated with data from the input stream. Consequently the input stream and `b` may be in an inconsistent state. It is strongly recommended that the stream be promptly closed if an I/O error occurs.

**Overrides:**

`readNBytes` in class `InputStream`

**Parameters:**

`b` - the byte array into which the data is read

`off` - the start offset in `b` at which the data is written

`len` - the maximum number of bytes to read

**Returns:**

the actual number of bytes read into the buffer

## skip

```
public long skip(long n)
```

Skips `n` bytes of input from this input stream. Fewer bytes might be skipped if the end of the input stream is reached. The actual number `k` of bytes to be skipped is equal to the smaller of `n` and `count-pos`. The value `k` is added into `pos` and `k` is returned.

**Overrides:**

`skip` in class `InputStream`

**Parameters:**

n - the number of bytes to be skipped.

**Returns:**

the actual number of bytes skipped.

**See Also:**

`InputStream.skipNBytes(long)`

## available

```
public int available()
```

Returns the number of remaining bytes that can be read (or skipped over) from this input stream.

The value returned is `count - pos`, which is the number of bytes remaining to be read from the input buffer.

**Overrides:**

`available` in class `InputStream`

**Returns:**

the number of remaining bytes that can be read (or skipped over) from this input stream without blocking.

## markSupported

```
public boolean markSupported()
```

Tests if this `InputStream` supports mark/reset.

**Overrides:**

`markSupported` in class `InputStream`

**Implementation Requirements:**

The `markSupported` method of `ByteArrayInputStream` always returns `true`.

**Returns:**

true

**Since:**

1.1

**See Also:**

`InputStream.mark(int)`, `InputStream.reset()`

## mark

```
public void mark(int readAheadLimit)
```

Set the current marked position in the stream. ByteArrayInputStream objects are marked at position zero by default when constructed. They may be marked at another position within the buffer by this method.

If no mark has been set, then the value of the mark is the offset passed to the constructor (or 0 if the offset was not supplied).

Note: The `readAheadLimit` for this class has no meaning.

**Overrides:**

`mark` in class `InputStream`

**Parameters:**

`readAheadLimit` - the maximum limit of bytes that can be read before the mark position becomes invalid.

**Since:**

1.1

**See Also:**

`InputStream.reset()`

## reset

```
public void reset()
```

Resets the buffer to the marked position. The marked position is 0 unless another position was marked or an offset was specified in the constructor.

**Overrides:**

`reset` in class `InputStream`

**See Also:**

`InputStream.mark(int)`, `IOException`

## close

```
public void close()
           throws IOException
```

Closing a `ByteArrayInputStream` has no effect. The methods in this class can be called after the stream has been closed without generating an `IOException`.

**Specified by:**

`close` in interface `AutoCloseable`

**Specified by:**

`close` in interface `Closeable`

**Overrides:**

`close` in class `InputStream`

**Throws:**

`IOException` - if an I/O error occurs.

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.